

Aula 9

- A interface RS-232C
 - Estrutura das tramas
 - Codificação dos sinais
 - Sincronização de relógio
 - Tolerância na frequência dos relógios do emissor e do recetor
- *Device drivers*
 - Princípios gerais
 - Exemplo da estrutura de um *device driver* para microcontrolador

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

Introdução

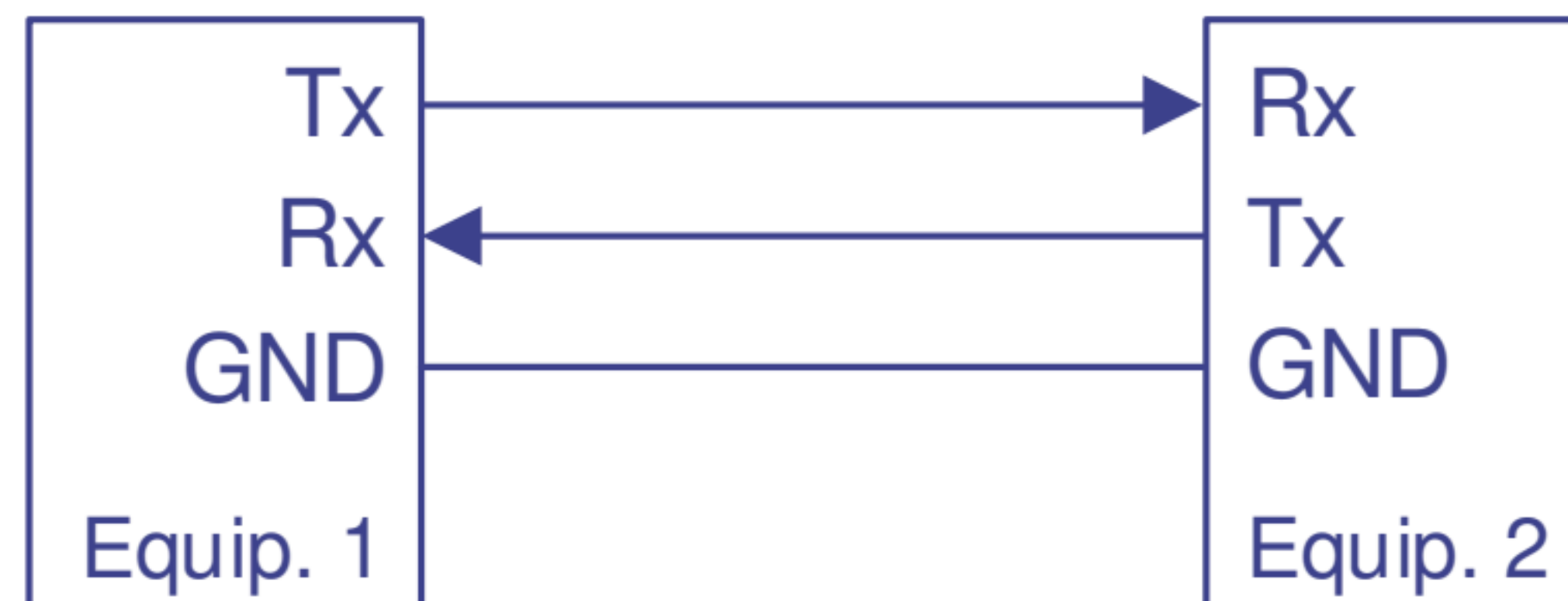
forma mais simples de meter dois computadores a comunicar

- RS-232C – Standard (1969) para comunicação série assíncrona entre um Equipamento Terminal de Dados (DTE, e.g. computador) e um Equipamento de Comunicação de Dados (DCE, e.g. Modem)
- Por ser um modo de comunicação série muito fácil de implementar e de programar continua a ser muito usado em microcontroladores
- Apareceram no mercado conversores USB/RS-232C que permitem a ligação a PCs de equipamentos que utilizam RS-232C
- Permite comunicação bidirecional, *full-duplex*
- Transmissão orientada ao byte
- Na sua forma mais simples, a implementação da norma RS-232C requer apenas a utilização de 2 linhas de sinalização: TX e RX

comunicação nos dois sentidos

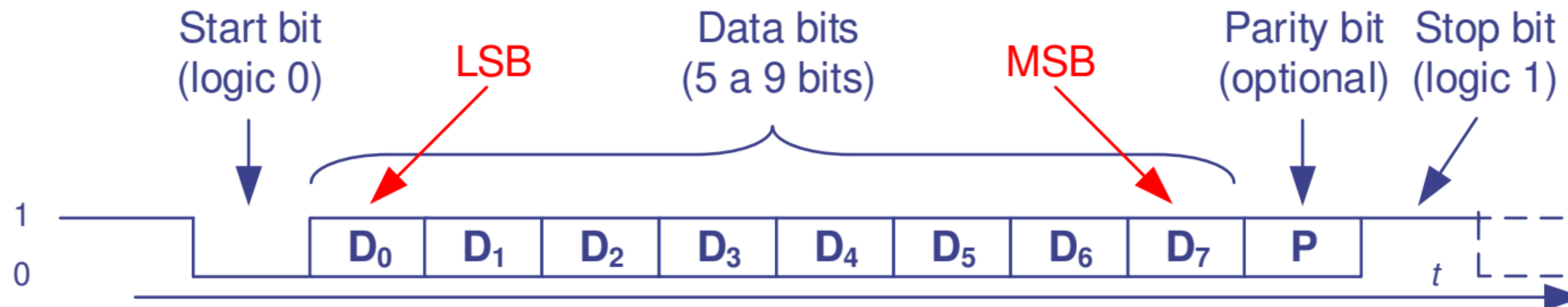
apenas ligações ponto a ponto

n tem clk →
comunicação assíncrona



Estrutura de uma trama RS-232C

comunicação começa pelo start bit (0) e acaba com o stop bit (a 1)



- **Parity Bit:** (opcional) quando utilizado, pode ser definido como paridade PAR ("even") ou paridade ÍMPAR ("odd")
 - É calculado pelo controlador de comunicação em cada trama:
 - **Paridade PAR:** o bit é obtido através do XOR de todos os bits do campo de dados. Exemplo: data bits – 10011000 -> parity bit: 1
 - **Paridade ÍMPAR:** o bit é obtido através do XNOR de todos os bits do campo de dados. Exemplo: data bits – 10011000 -> parity bit: 0
 - Permite detetar erros de comunicação sempre que ocorra um número ímpar de bits errados
- **Stop Bit(s):** podem ser utilizados 1 ou 2 bits
 - Coincidem com o estado de linha inativa (*idle*)
 - Proporcionam um intervalo de tempo de guarda mínimo entre a transmissão consecutiva de dois valores

Para detetar erros
↓
Calculado trama a trama

antes de enviar a trama

Geração / Receção da Trama RS-232C

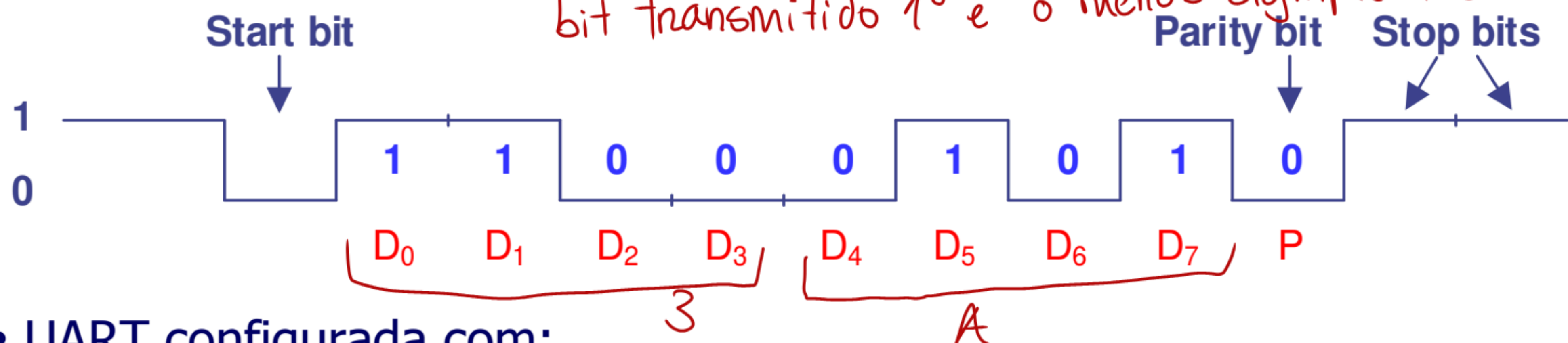
- A trama é gerada pelo controlador de comunicação série RS-232C, normalmente designado por **UART** (Universal Asynchronous Receiver/Transmitter) *(controlador que implementa o RS-232C)*
- Na transmissão:
 - A UART converte dados enviados em paralelo pelo processador em dados série
 - Adiciona os bits de "start", dados, *calcula e adiciona* paridade (se usada) e "stop", de acordo com a configuração que tiver sido feita
- Na receção: *→ verifica se houve erro (de paridade)*
 - A UART recebe a trama série, converte-a em paralelo e extrai a palavra de dados, disponibilizando-a num registo para poder ser lida pelo processador
- A transmissão/receção ocorre de forma assíncrona (não há transmissão nem recuperação do relógio)

Exemplos de trama RS-232C

- UART configurada com:

- 8 bits de dados, paridade par e 2 stop bits (8,E,2)

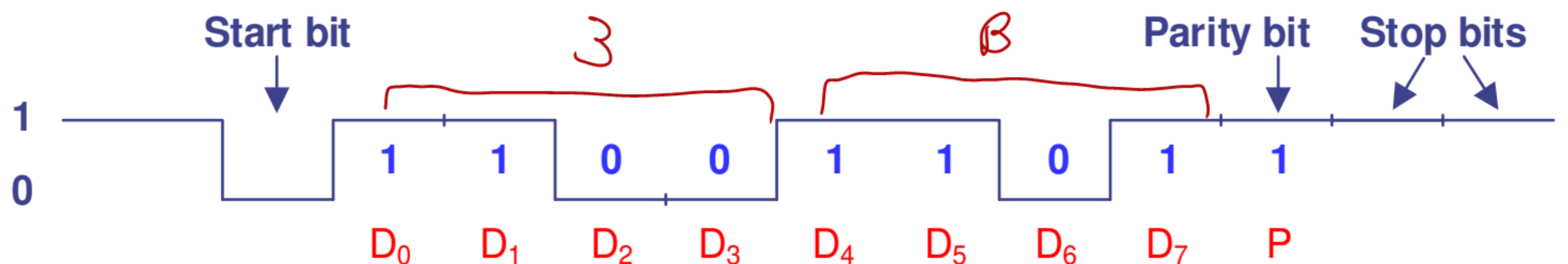
- Valor a transmitir: 0xA3



- UART configurada com:

- 8 bits de dados, paridade par e 2 stop bits (8,E,2)

- Valor a transmitir: 0xB3



← linha fica a 1 por dois clk cycles

Baudrate (taxa de transmissão)

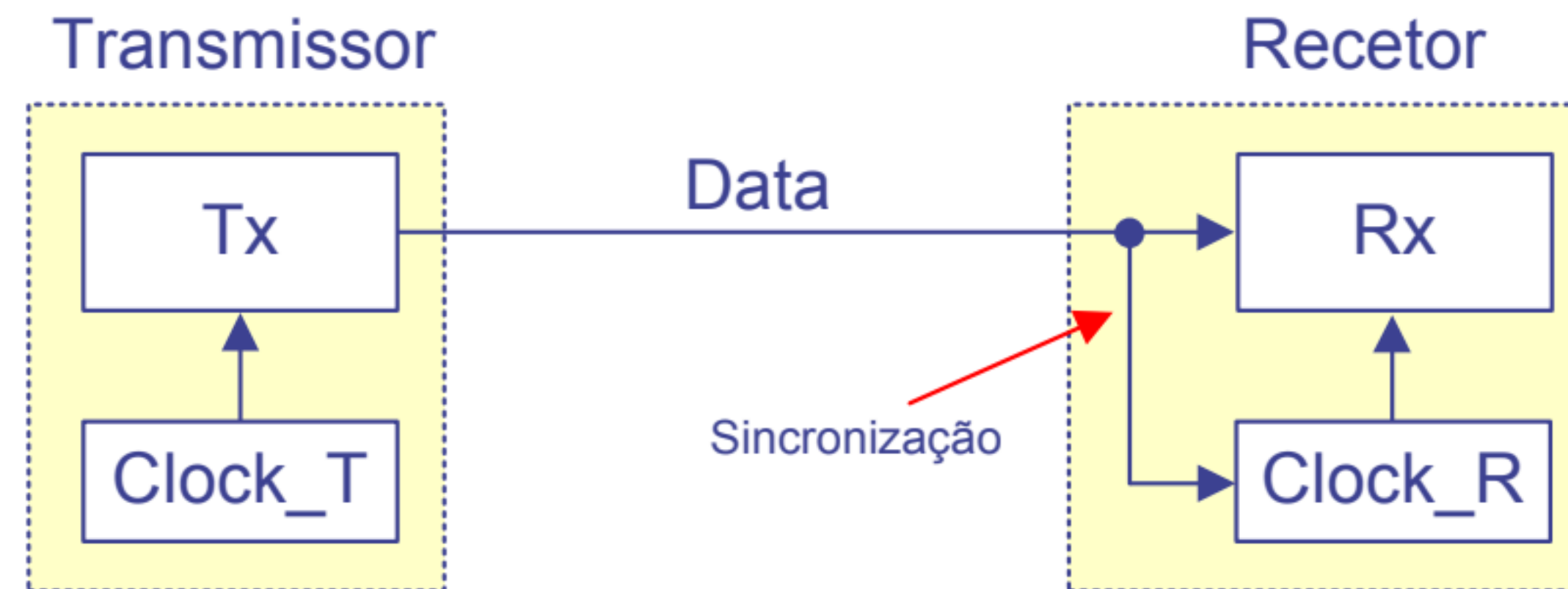
↪ 1 bit por ciclo de clk

- O **baudrate** é, genericamente, o número de símbolos transmitidos por segundo; a cada símbolo pode corresponder um ou mais bits de dados
- A taxa de transmissão de dados bruta (*gross bit rate*) corresponde ao número de bits transmitidos por segundo (bps) (o *baudrate* não deve ser confundido com *gross bit rate*)
- No caso do RS-232C a cada símbolo está associado um único bit, logo o *baudrate* e o *gross bit rate* coincidem
- Exemplos comuns de *baudrates* em RS-232C [bps]: 600, 1200, 4800, 9600, 19200, 38400, 57600, 115200, 230400
↪ 600 bits por segundo
↪ usamos no plerom
- No exemplo do slide anterior o número total de bits a serem transmitidos é 12
 - 1 start bit, 8 bits de dados, 1 bit de paridade, 2 stop bits
 $1 + 8 + 1 + 2$
 - considerando um *baudrate* de 57600 bps a transmissão completa de uma trama demora $\sim 208 \mu s$ (12 / 57600)
 - o bit rate líquido é $(8 * 57600) / 12$, i.e., 38400 bps
↪ 4 em 12 são overhead
 - o byte rate é: $38400 / 8 = 4800$ bytes/s
↪ baixo → usado para coisas de baixa frequência

Receção de uma trama

- Sincronização de relógio: **relógio implícito**

- Transmissão assíncrona
- O transmissor e o recetor têm relógios locais (independentes)



bit start faz recetor
sincronizar-se
para receber informação

- A sincronização do processo de receção de dados é assegurada **no início da receção de cada nova trama** (sinalizado pelo **start bit**: transição de "1" para "0" na linha após um período de inatividade, por exemplo depois da receção completa de uma trama)
- Este método deve ser robusto, dentro de certos limites, a diferenças de frequência entre os relógios do transmissor e do recetor
 - Imprecisão na geração do relógio
 - Constante de divisão dos timers (que geram o relógio) não inteiras

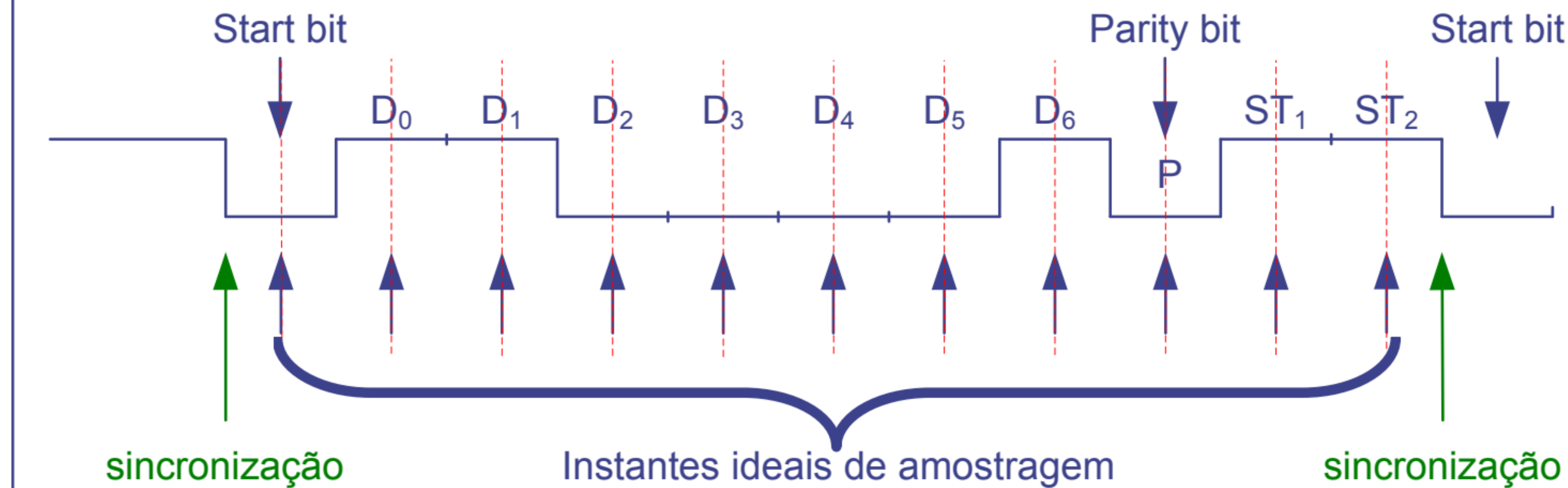
} dois relógios
não vão
ter a
mesma freq.

Só funciona se tiverem frequências próximas !! (diferença entre clocks < 3%)

Receção de dados

Parâmetros dos dois lados têm de ser o mesmo

- Para que a comunicação se processe corretamente, o transmissor e o recetor têm que estar **configurados com os mesmos parâmetros**:
 - Estrutura da trama: **nº de bits de dados, tipo de paridade, número de stop bits**
 - **Baudrate** (relógios com a mesma frequência) *(diferença menor que 3%)*
- O recetor deve sincronizar-se pelo flanco negativo (transição do nível lógico "1" para o nível lógico "0") da linha (Start bit) e, idealmente, **fazer as leituras a meio do intervalo reservado a cada bit**
- Exemplo da receção do valor 0x43: estrutura da trama 7, 0, 2 (7 data bits, odd parity, 2 stop bits)

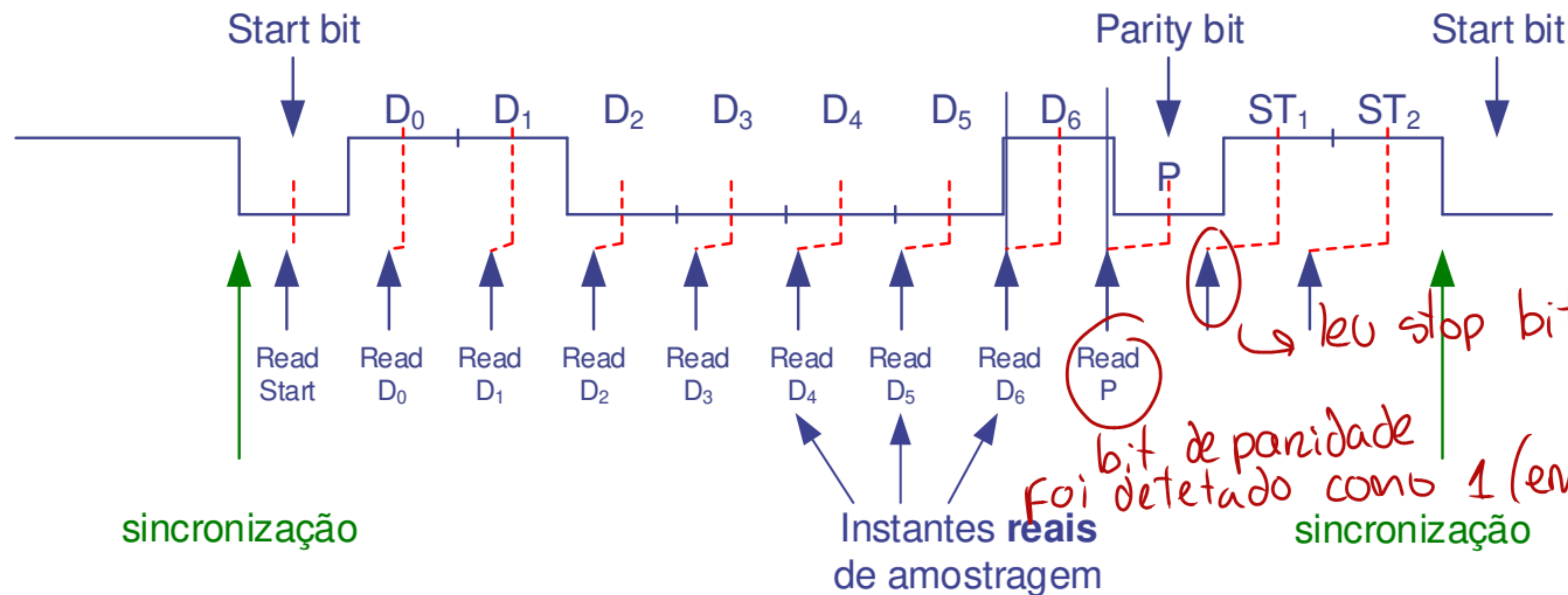


recetor, a partir do start bit, armazena os bits que aparecem na linha sempre em intervalos de n -tempo, de forma "cega"

Sincronização

exemplo: freq do recetor é maior

- Entre instantes de sincronização o desvio dos relógios depende da estabilidade/precisão dos relógios do transmissor e do recetor
- Exemplo em que a receção não é corretamente efetuada devido a um desvio da frequência dos relógios do transmissor e do recetor

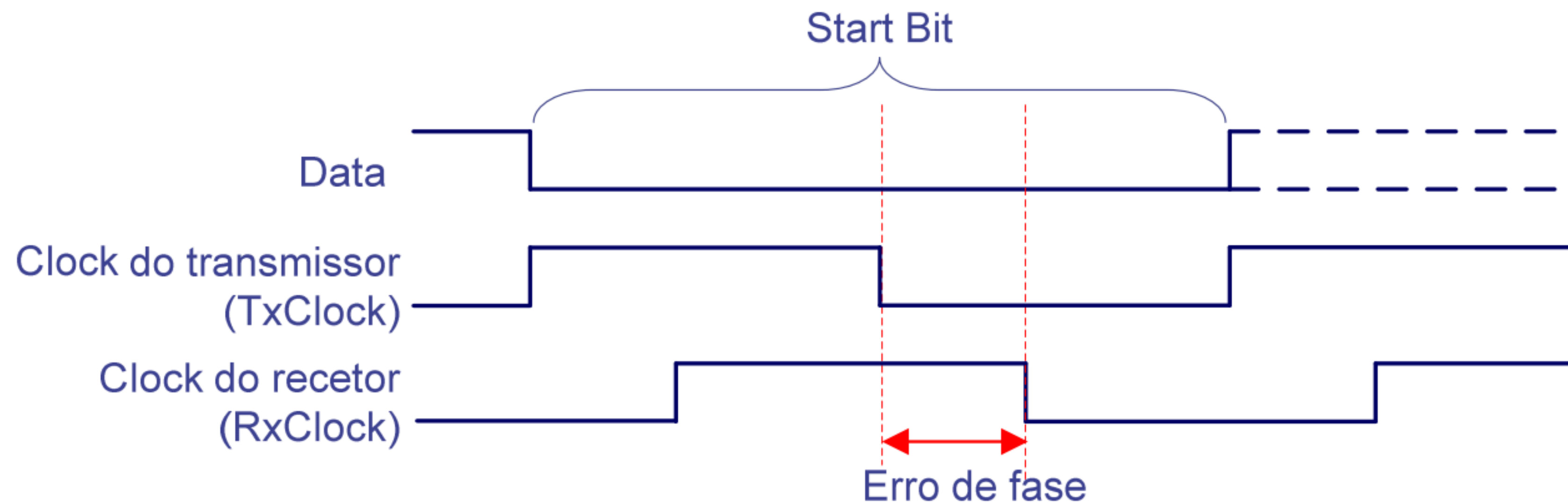


- Neste exemplo o recetor detetaria dois erros:
 - **Erro de paridade** (se o bit D6 for detetado como 1): o bit de paridade devia ser 0 e é lido como 1
 - **Erro de framing**: é detetado o nível lógico 0 no instante em que era esperado um stop bit (nível lógico 1)

apanecem no status

Can resolve

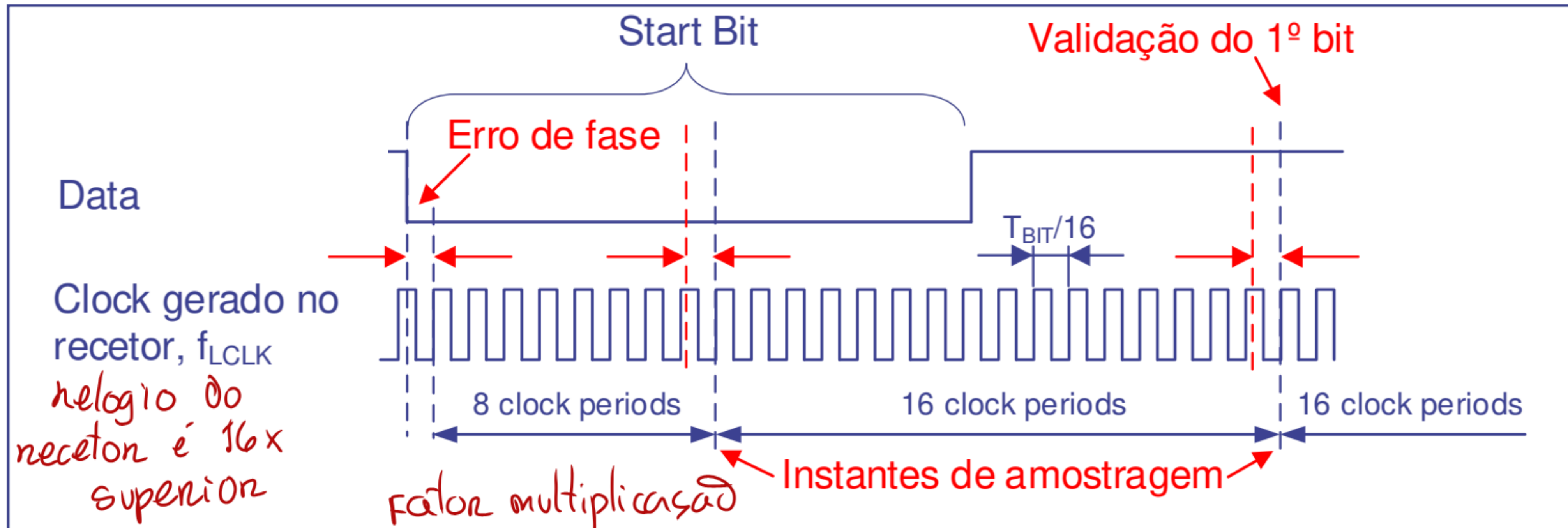
Sincronização



- Mesmo que a frequência dos relógios do transmissor e do recetor seja a mesma, **subsiste o erro de fase** que pode impedir a correta validação da informação (idealmente a meio do "tempo de bit")
- Sincronizar a fase do relógio do recetor com a do transmissor é tecnicamente complicado
- Em vez disso, é mais simples gerar no recetor um relógio com uma frequência N vezes superior ao relógio do transmissor e sincronizar a receção a partir desse relógio (designado a seguir por f_{LCLK})

com 1 contador contamos os N ciclos e lêmos a linha

Sincronização



- Por exemplo, se $N = 16$, o erro de fase máximo desse relógio, relativamente ao aparecimento do start, é $T/16$, em que T é o período do relógio do transmissor (ou seja, o "tempo de bit", T_{BIT})
- O erro de fase mantém-se até ao fim da receção da trama corrente, mas os instantes de validação estão bem definidos:
 - "Start bit", validado ao fim de 8 ciclos de relógio
 - Restantes bits validados a cada 16 ciclos de relógio

erro de fase é um período do relógio local $\rightarrow$ maior fator de multiplicação = erro menor

Sincronização

- O relógio local (**LCLK**) deverá então ter, idealmente, uma frequência igual a:

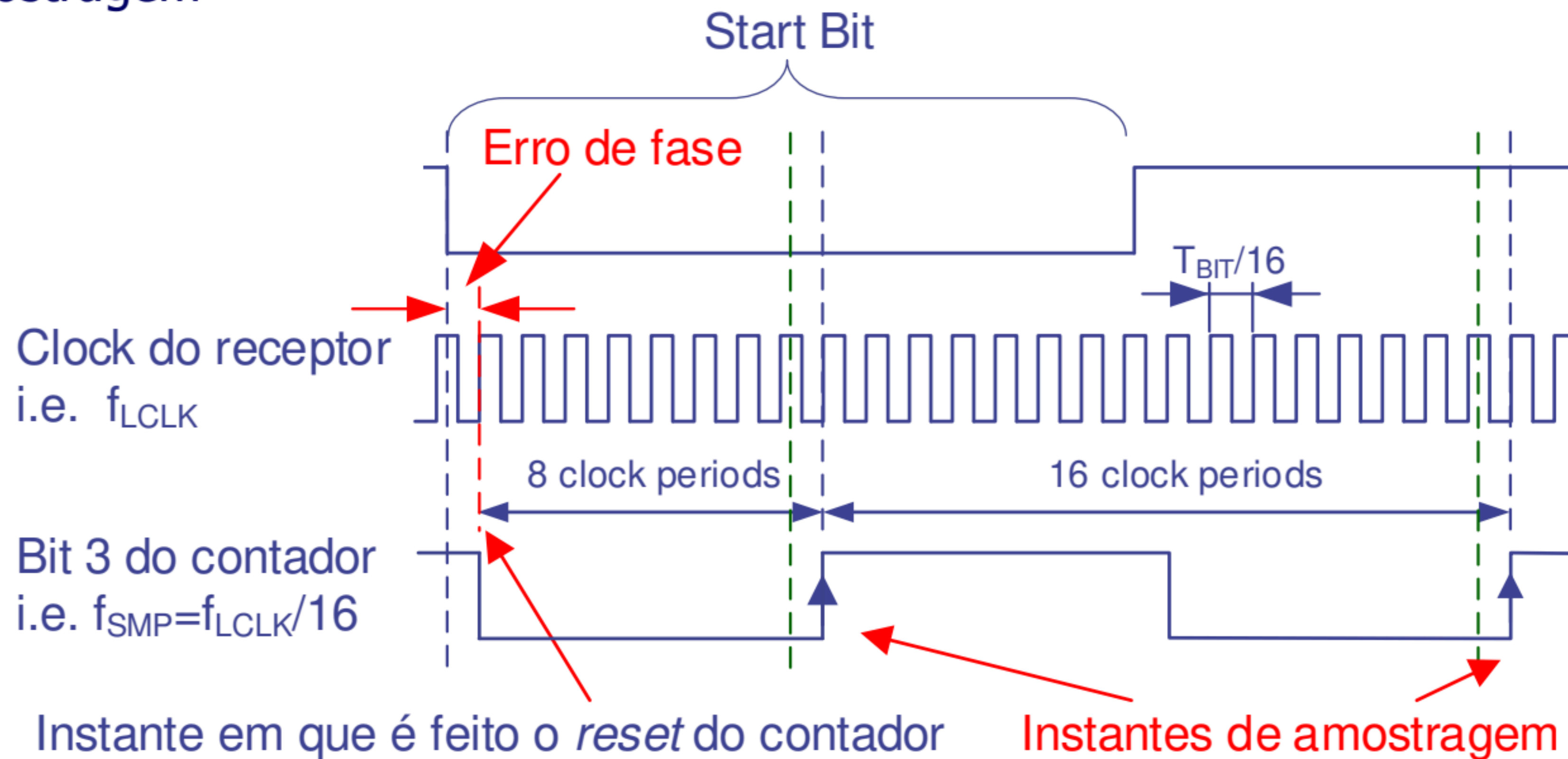
$$f_{\text{LCLK}} = N * f_{\text{TCLK}}$$

em que $f_{\text{TCLK}} = 1/T_{\text{BIT}}$ é a frequência do relógio de transmissão

- N é normalmente designado por **fator de sobreamostragem**
- Valores típicos de N: 4, 16, 64
- Esse relógio não é sincronizado com o sinal da linha, logo impõe um erro de fase (que é sempre inferior a um período, $\Delta_1 < T_{\text{LCLK}}$)
- Utilizando um relógio com N=16 o erro de fase máximo é **$T_{\text{BIT}}/16$** . Para N=64, o erro de fase máximo será **$T_{\text{BIT}}/64$**
- A utilização de fatores de sobreamostragem elevados nem sempre é possível (frequência da fonte de relógio disponível, constante de divisão do timer)

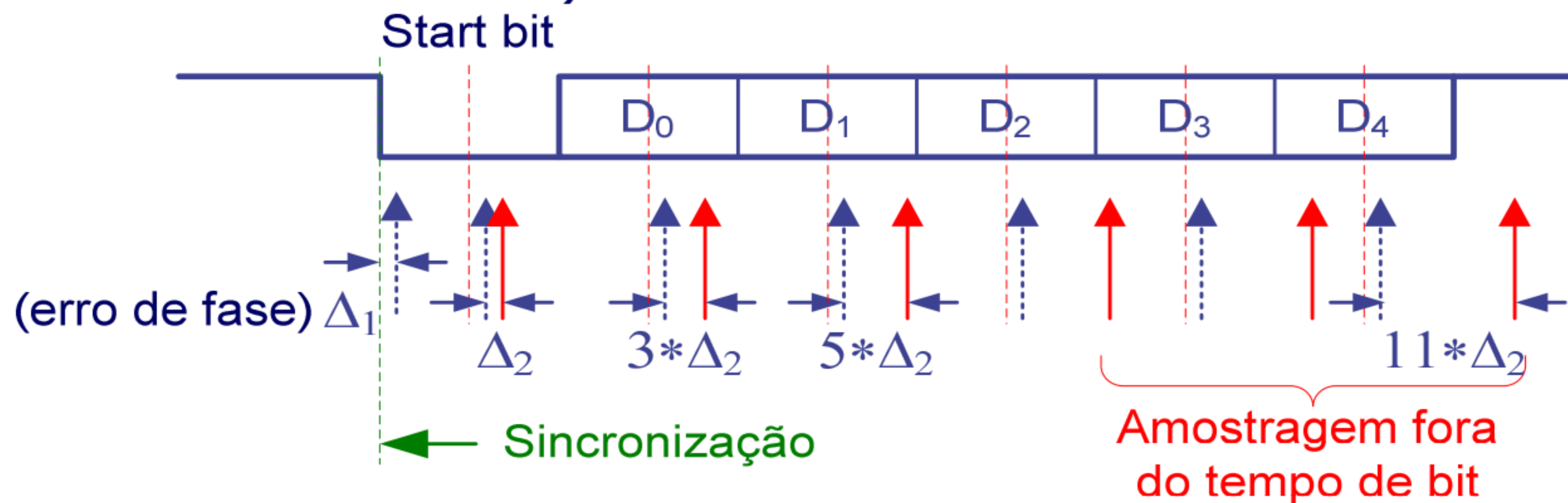
Sincronização – exemplo de implementação

- Considerando um fator de sobreamostragem (N) de 16, o tempo de bit equivale a 16 períodos do relógio local ($16T_{LCLK}$). O erro de fase máximo é $T_{BIT}/16$
- f_{SMP} é a frequência usada pelo recetor para a amostragem de cada bit da trama
- Usando um contador de 4 bits (com o relógio f_{LCLK}) como divisor de frequência por 16 (i.e. $f_O = f_{SMP} = f_{LCLK}/16$) a sincronização é trivial: basta fazer o reset (síncrono) desse contador quando é detetado o start bit
- As transições ascendentes do bit 3 do contador (MSBit) definem os instantes de amostragem



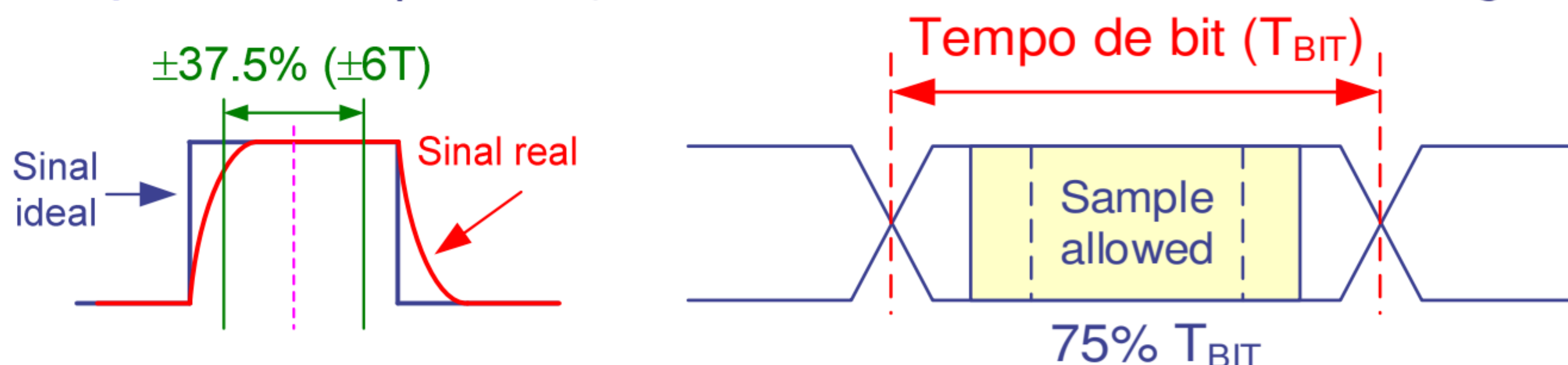
Sincronização – erro do instante de amostragem

- Os erros nos instantes de amostragem podem ter duas causas distintas:
 - **Erro de fase** (Δ_1): erro cometido ao determinar o instante inicial de sincronização
 - **Erro provocado por desvio de frequência** (Δ_2): a frequência dos relógios do transmissor e do recetor não são exatamente iguais (e.g. tolerância dos cristais de quartzo dos osciladores, constantes de divisão dos timers). Este erro é cumulativo e proporcional ao comprimento da trama (no caso da figura abaixo, no bit D_4 o erro é $11 * \Delta_2$)
- O efeito cumulativo destas fontes de erro pode originar erros na receção (como observado no slide 9)



Máximo desvio de frequência entre emissor e recetor

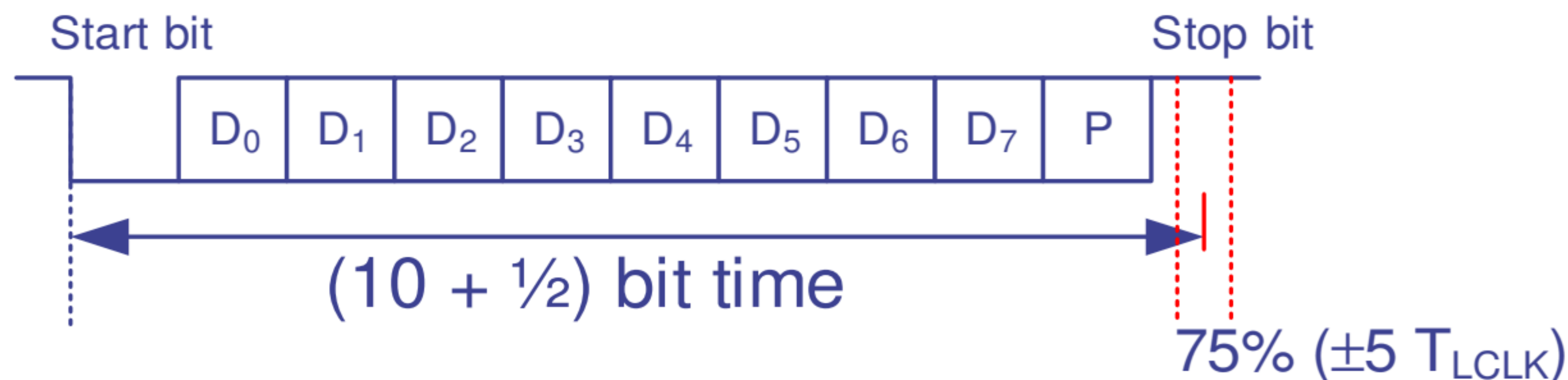
- Considerando como zona segura de amostragem do bit:
 - $\pm 37,5\%$** do tempo de bit, em torno do instante ideal de amostragem



- Considerando um fator de sobreamostragem (N) de 16, o tempo de bit equivale a 16 períodos do relógio local (**$T_{\text{BIT}} = 16T_{\text{LCLK}}$**)
- Assim, o desvio máximo admitido no instante de amostragem de um bit é de **$\pm 6T_{\text{LCLK}}$** ($\pm 0.375 * 16$ ciclos do relógio local)
- Como há um erro de fase intrínseco de, no máximo, de **$1T_{\text{LCLK}}$** então o desvio máximo aceitável, resultante da diferença de frequência dos relógios, é **$-6T_{\text{LCLK}}$** a **$+5T_{\text{LCLK}}$** (em todos os instantes de amostragem)
 - Por uma questão de simplificação consideramos um desvio máximo de **$\pm 5T_{\text{LCLK}}$**

Máximo desvio de frequência entre emissor e recetor

- Consideremos então o caso em que pretendemos amostrar uma das tramas mais longas (start bit, 8 data bits, bit de paridade e 1 stop bit)
- É necessário garantir que o último bit da trama é amostrado dentro da zona segura



- Para amostrar esta trama serão necessários $10.5 \times 16 = 168$ períodos do relógio local (fator de sobreamostragem de 16)
- Assim, a máxima discrepância que poderá ser tolerada entre os relógios do transmissor e do recetor é $\Delta T = \pm 5 / 168 \approx \pm 3.0\%$

Máximo desvio de frequência entre emissor e recetor

- **Exercício:** considere-se uma comunicação RS-232C com os seguintes parâmetros: 115200 bps, 8 data bits, parity bit, 1 stop bit. Supondo um fator de sobreamostragem de 16, calcule o valor mínimo e máximo da frequência do relógio do recetor.
- Valor ideal da frequência:

$$T_{LCLK} = \frac{1}{(16 * 115200)} \gg f_{LCLK} = 1.843.200 \text{ Hz}$$

- Gama admissível de frequência do relógio local:

$$T_{LCLK} = \frac{(1 \pm 0.03)}{(16 * 115200)} \gg f_{LCLK} \in [1.798.515, 1.900.206] \text{ Hz}$$

- *Device drivers*

- Princípios gerais
- Exemplo da estrutura de um *device driver* para microcontrolador

Device Drivers

- O **número** de **periféricos** existentes é muito **vasto**:
 - Teclado, rato, placas (gráfica, rede, som, etc.), disco duro, *pen drive*, scanner, câmara de vídeo, etc.
- Estes periféricos apresentam características distintas:
 - **Operações suportadas**: leitura, escrita, leitura e escrita
 - **Modo de acesso**: carater, bloco, etc.
 - **Representação da informação**: ASCII, UNICODE, *Little/Big Endian*, etc.
 - **Largura de banda**: desde alguns bytes/s até MB/s
 - **Recursos utilizados**: portos (I/O, *memory mapped*), interrupções, DMA, etc.
 - **Implementação**: diferentes dispositivos de uma dada classe podem ser baseados em implementações distintas (e.g. diferentes fabricantes, diferentes modelos) com reflexos profundos na sua operação interna
- Esta diversidade torna impossível para o sistema operativo tratar todos os dispositivos de forma direta e uniforme

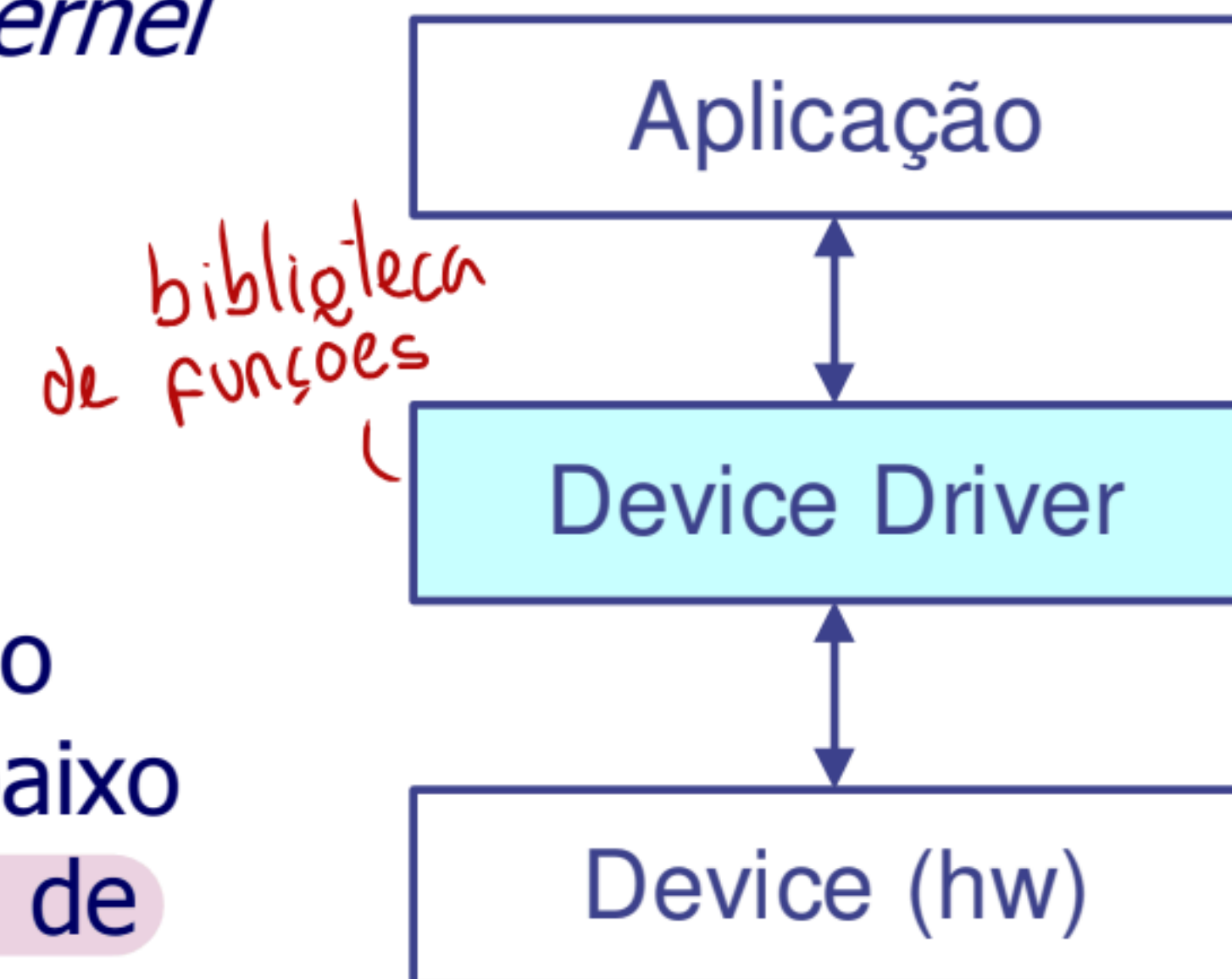
Device Drivers ^{, software} ^{tradutor entre software e hardware} ^{tem de conhecer todos os detalhes do hardware}

- As aplicações/Sistemas Operativos (SO) não podem conhecer todos os tipos de dispositivos passados, atuais e futuros com um nível de detalhe suficiente para realizar o seu controlo a baixo nível
- **Solução:** introduzir uma camada de abstração que permita aceder aos dispositivos de forma uniforme e independente da sua implementação
- **Device driver**
 - Um programa que permite a outro programa (aplicação, SO) interagir com um dado dispositivo de hardware
 - Implementa a camada de abstração e lida com as particularidades do dispositivo controlado
 - Normalmente fornecido pelo fabricante, pois requer conhecimento detalhado do dispositivo
- Aspectos-chave:
 - **Abstração:** esconde a complexidade do hardware
 - **Uniformização de acesso:** diferentes dispositivos da mesma classe são acedidos da mesma forma ^{→ mesma interface de acesso}
 - **Independência entre aplicações/SO e o hardware:** facilita a compatibilidade e a portabilidade do software

*rato e teclado
são HID*

Device Drivers – Princípios gerais

- O acesso, por parte das aplicações, a um *device driver* é diferente num sistema embutido e num sistema computacional de uso geral (com um Sistema Operativo típico, e.g. Linux, Windows, Mac OS)
- Sistemas embutidos: as aplicações acedem diretamente aos *device drivers*
 - Estes sistemas normalmente não têm *kernel* (núcleo de um SO) ou têm um *kernel* mínimo, permitindo que o software de aplicação interaja diretamente com o hardware
 - A ausência de uma camada de abstração complexa permite respostas rápidas e baixo *overhead*, fatores críticos para sistemas de tempo real

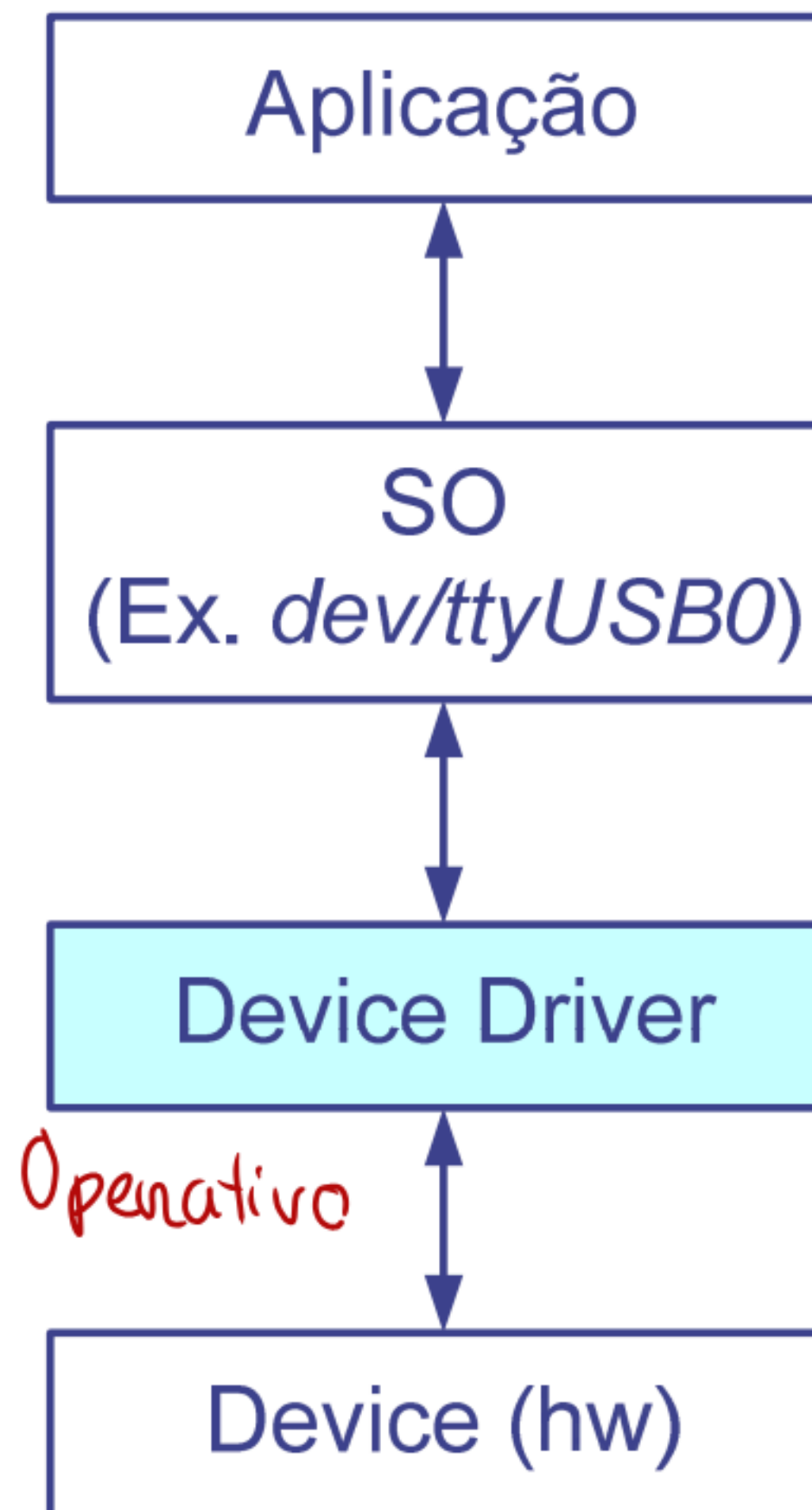


Device Drivers – Princípios gerais

- Nos sistemas operativos típicos (e.g., Linux, Windows, macOS), as aplicações não interagem diretamente com os *device drivers*: as aplicações fazem chamadas ao sistema (*system calls*) quando precisam de interagir com o hardware

- O kernel do SO recebe estas chamadas e encaminha-as para o *device driver* adequado
- O *device driver* executa as operações específicas de acesso ao hardware (registos, interrupções, DMA, ...)

Quem interage com o Device Driver é o Sistema Operativo



Fluxo de interação com dispositivos (sistemas com SO)

- Aplicação não acede diretamente ao hardware
- Fluxo típico de execução:
 - Aplicação → System Call → Kernel → Device Driver → Hardware
- Passos:
 - A aplicação faz um pedido (ex: ler ficheiro) *as hardware pen, disco duro, etc*
 - O *kernel* recebe o pedido através da *system call*
 - O *kernel* encaminha para o *device driver* adequado
 - O *driver* comunica com o hardware
 - O resultado percorre o caminho inverso
- Vantagens:
 - Segurança
 - Abstração
 - Portabilidade

Device Drivers – Princípios gerais

Aplicações e kernel funcionam em ambientes separados

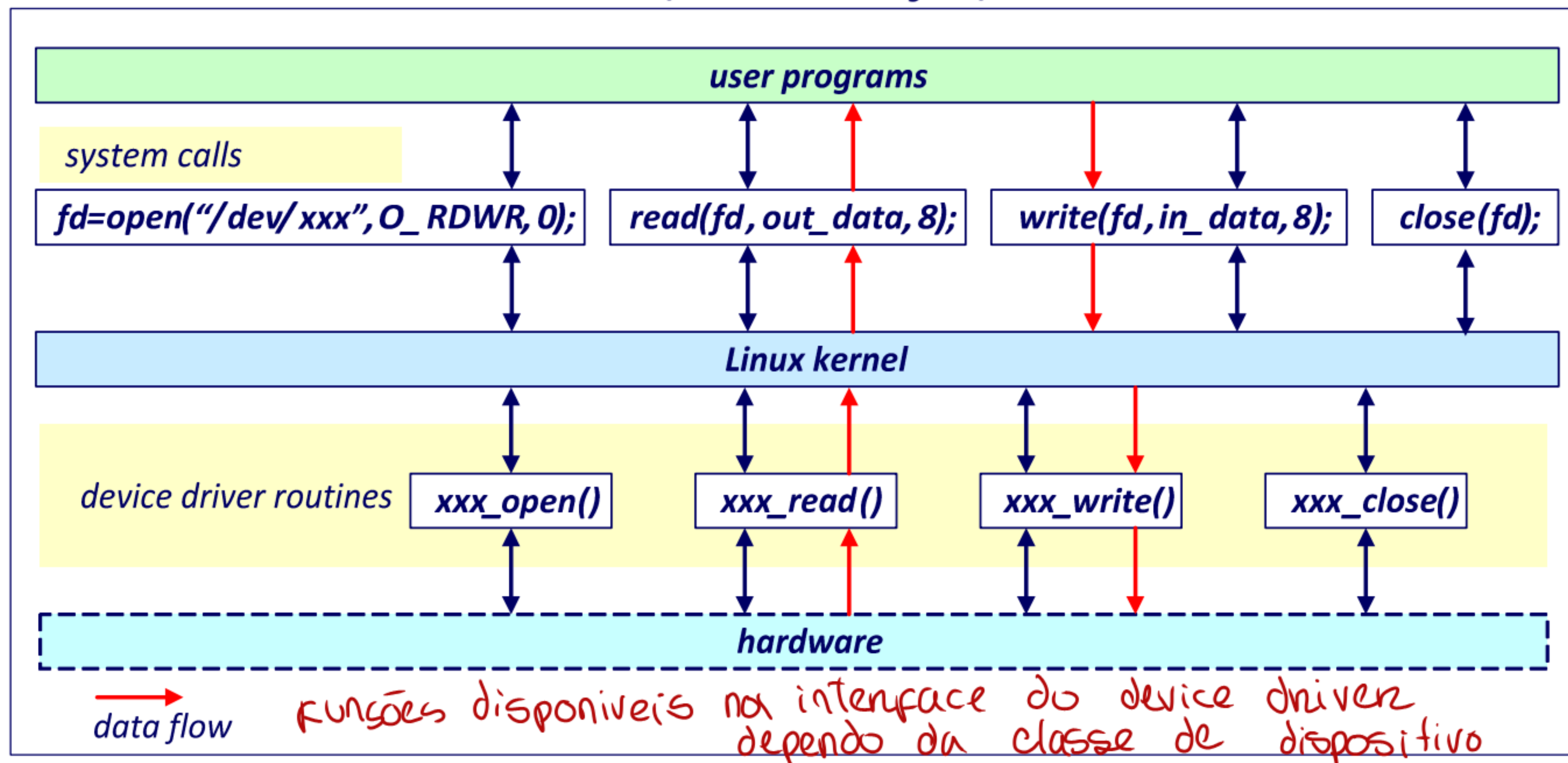
- A separação em camadas é feita através de dois ambientes de execução distintos: "user space" e "kernel space"
 - "Espaço de utilizador" (User Space): onde as aplicações comuns são executadas, sem acesso direto ao hardware, o que evita que erros nas aplicações afetem o funcionamento do sistema operativo
 - "Espaço de Kernel" (Kernel Space): onde o sistema operativo e os *device drivers* operam, com acesso privilegiado ao hardware, permitindo ao sistema controlar os recursos físicos da máquina de forma segura
- Vantagens: segurança, estabilidade
 - Aplicações maliciosas ou com erros não podem aceder diretamente ao hardware ou interferir com o sistema operativo
 - Erros em aplicações no espaço de utilizador não comprometem o funcionamento do sistema como um todo (o *kernel* continua a funcionar normalmente)

erros de aplicações
n podem
estragar o
sistema
operativo

erros em drivers resulta em tragédia porque
correm em Kernel Space

Device Drivers – Princípios gerais

- O Sistema Operativo especifica classes de dispositivos e, para cada classe, uma interface que estabelece como é realizado o acesso a esses dispositivos
 - A função do *device driver* é traduzir as chamadas realizadas pela aplicação/SO em ações específicas do dispositivo
 - Exemplos de classes de dispositivos: interface com o utilizador, armazenamento de massa, comunicação, ...



Exemplo: leitura de dados de um disco

- Código da aplicação:
`read(fd, buffer, size);`
- O que acontece internamente:
 - A aplicação invoca a *system call* **read()**
 - O *kernel* valida o pedido
 - O *kernel* chama o driver de disco
 - O *driver*:
 - envia comandos ao controlador do disco
 - pode usar DMA para transferência de dados (normalmente é usado DMA)
 - O hardware lê os dados
 - O *driver* notifica o *kernel* (ex: interrupção)
 - O *kernel* devolve os dados à aplicação
- A aplicação não precisa de conhecer:
 - registros
 - protocolos do dispositivo
 - detalhes do hardware

Classificação e função dos *device drivers*

- O sistema operativo organiza os dispositivos em classes (interface com o utilizador, armazenamento de massa, comunicação, etc.) e define
 - Para cada classe, define uma interface padrão de acesso ^{definida pelo SO}
 - O *device driver* implementa essa interface, traduzindo as chamadas do Sistema Operativo em ações específicas sobre o hardware
- Existem diferentes tipos de *device drivers*, consoante a natureza do dispositivo:
 - *Drivers* de carater: acedem aos dados como um fluxo de bytes (ex: teclado, rato, porta série) → coisas de baixo débito
 - *Drivers* de bloco: acedem aos dados em blocos (ex: discos, pen drives) → desenhados para aceder a discos (transferir blocos de dados)
 - *Drivers* de rede: tratam pacotes de dados (ex: interfaces Ethernet, Wi-Fi) ^{ou pacote}
→ Drivers de bloco mas com destino e os blocos são lidos só uma vez (informação é consumida)
 - Cada tipo de *driver* tem mecanismos de acesso e interfaces específicas no Sistema Operativo

Exemplo de um *device driver*: comunicação série

- Admitindo que é fornecida uma biblioteca que apresenta a seguinte interface:

- `void comDrv_init(int baudrate, char dataBits, get char char parity, char stopBits);`
- `char comDrv_getc(void);` // read a character
- `void comDrv_putc(put char char ch);` // write a character
- `void comDrv_close(void);`

- **Do ponto de vista da aplicação:**

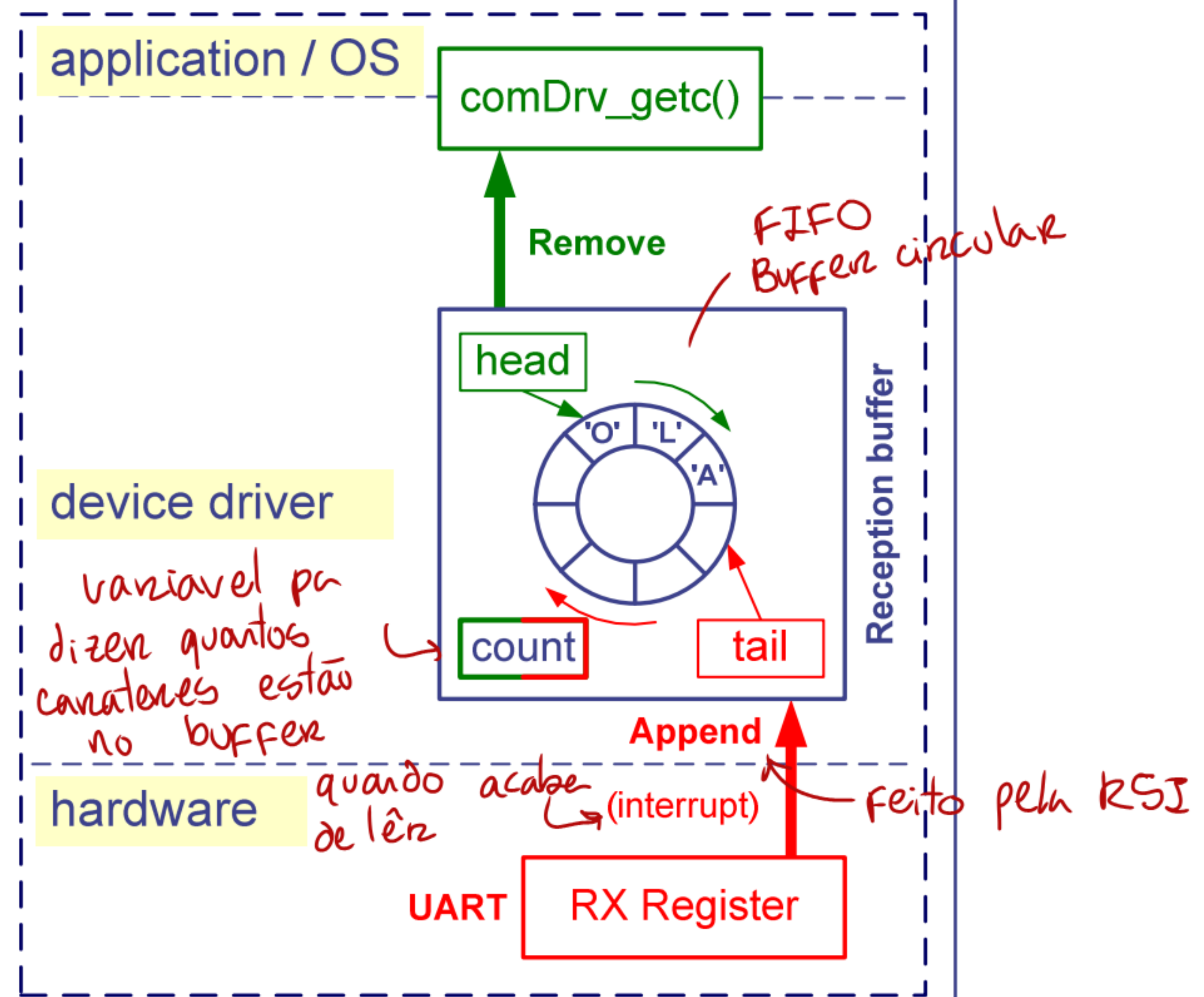
- Do ponto de vista funcional é relevante qual o modelo/fabricante do dispositivo de comunicação série?
- Se o dispositivo de comunicação for substituído por outro com arquitetura interna distinta, sendo fornecida uma biblioteca com interface compatível, é necessário alterar a aplicação? *Não, apenas é preciso alterar a biblioteca e recompilar o código*

Exemplo de um *device driver* para microcontrolador

- Aspectos-chave da implementação de um *device driver* para uma UART RS232 (*Universal Asynchronous Receiver Transmitter*) para executar num sistema com microcontrolador (i.e., sem sistema operativo)
- Princípio de operação
 - Desacoplamento da transferência de dados entre a UART e a aplicação, realizada por meio de FIFOs (um FIFO de transmissão e um de receção). Do ponto de vista da aplicação:
 - A **transmissão** consiste em copiar os dados a enviar para o FIFO de transmissão do *device driver*
 - A **receção** consiste em ler os dados recebidos que residem no FIFO de receção do *device driver*
 - A transferência de dados entre os FIFOs e a UART é realizada por interrupção, i.e., sem intervenção explícita da aplicação
 - Um FIFO pode ser implementado através de um *buffer* circular

Princípio de operação – receção

- **"tail"** – posição do buffer circular onde a **rotina de serviço à interrupção escreve** o próximo carácter lido da UART
- **"head"** – posição do buffer circular de onde a função **comDrv_getc()** lê o próximo carácter
- **"count"** – número de caracteres residentes no buffer circular (ainda não lidos pela aplicação)
- **O acesso à variável "count" tem que ser feito numa secção crítica do código. Porquê?**



Princípio de operação – transmissão

- **"tail"** – posição do buffer circular onde a função **comDrv_putc()** escreve o próximo carácter
- **"head"** – posição do buffer circular de onde a **rotina de serviço à interrupção** lê o próximo carácter a enviar para a UART
- **"count"** – número de caracteres residentes no buffer circular (ainda não enviados para a UART)
- **O acesso à variável "count" tem que ser feito numa secção crítica do código. Porquê?**

